

micro1 Workflow Evals: Team Playbook v5

Find a real workflow, write it as instructions, validate it, get approved by micro1 on the first try.

How to use this

- **First time reading this?** Read the whole thing top to bottom. The five steps are the path; the reference sections are there when you need detail.
 - **Read an earlier version already?** Read the Change log just below, then skim. You do not need to reread everything.
-

Change log

v5 (current version). Fixed how the two 1-to-7 rating fields are described, because they were easy to mix up.

- Feather has two separate 1-to-7 fields that point in opposite directions. **Workflow difficulty:** 1 is extremely easy, 7 is extremely difficult, and you just report yours honestly. **Rate the experience and outcome (the Codex test field):** 1 is horrible or completely unsuccessful, 7 is perfect, and here we want a low score (around 1 to 3), because a low score means the model could not do the task well, which is the goal. Earlier wording blurred these; they are now separated everywhere, using the exact field names.
- The Step 3 validator now also estimates the **Workflow difficulty** score, alongside the expected outcome score, so you have a starting point for both rating fields.
- Refined Rule 4 in the validator so it no longer fails prompts that create a new sheet, board, or project. Naming the minimum information each item must contain is enough; you do not have to design the full structure.
- Expanded the "be specific enough to avoid a revision" section and validator Rule 9 from a batch of reviewer feedback. They now cover: bounding any "review/search" step, naming the exact account or property, giving reference sources, saying what to extract from inputs, exact date logic, update-in-place versus create-new, output format (columns, date formats, sample row, filename), judgment criteria, and draft-versus-live for messages and invites. Also added a note that the occupation field reads best short and specific.
- The Step 3 validator now writes each fix out in full, with square brackets for the values you fill in (property names, folders, people), converts any relative date range like "last 30 days" to exact calendar dates, flags any text that reads AI-written and rewrites it to sound human, and returns a full fixed version of your prompt. In that fixed version, removed text is shown with `~~strikethrough~~` and added text inside `{{ curly braces }}`, so you can see the changes in any plain text editor. Fill in the brackets and keep the wording in your own casual voice.

- Added a "balance" section, the counterweight to the specificity guidance: be specific about what affects scoring (completion criteria, research boundaries, batch limits, code-change permission), but leave open what tests the model's judgment (how it organizes work, ticket structure, extra fields). The test is whether a capable junior would normally decide it themselves. The validator now applies this balance so it does not flag legitimately open areas.
- Fixed the validator so it no longer fails Rule 4 or Rule 9 for an intentionally open structure (like a Jira ticket layout) when the minimum information is stated. The balance test now runs as a gate before any specificity fail, Rule 4 passes open new-system structures with a Jira example, and Rule 9 only requires details that affect scoring. A reviewer confirmed leaving such structure open is fine when it matches how you would authentically assign the work.
- Added a blunt directive at the top of the validator and a "make the minimum concrete" rule to the balance section. The lesson: when you leave a structure open, name a few concrete required fields (not "related details", which reads as vague to the validator and reviewers), then invite the model to add the rest. A clear minimum plus open structure passes cleanly and still tests judgment; if the validator still over-flags an open structure with a minimum stated, that is a false positive and the reviewer's guidance wins.

v4. The reviewer's corrections, still in force.

- **Write prompts as instructions, no "I" statements.** Use "Review the records", "Create the folder", "Update the list", not "I review" or "I create". This was the reviewer's main and most common note.
- **Fill in every Feather field and mark the task Completed** (top right) before sharing.
- **Actually test the prompt in Codex and confirm the model cannot do it well** (a low score on the "Rate the experience and outcome" field, around 1 to 3). Add difficulty if the model does it well.
- **Fill the environment field:** the local professional environment and the resources the agent needs. The reviewer specifically values this.
- **The prompt only ever runs on your own workspace**, so reference your real files and apps by name. You do not need to upload or link them. Just keep client data and credentials out of the text.
- **Every line in "The bar" tells you what to actually do**, and the Step 3 validator returns plain-language results with a worked example.

The path, in five steps

1. List your tasks (simple)
2. Turn one task into a prompt, written as instructions (real example inside)
3. Validate it with our self-QA prompt
4. Run it in Codex, and in Claude side by side
5. Fill every Feather field, mark Completed, and ping the team

Reference sections follow: tools, real SME examples by role, the no-real-workspace path, the full worked example with every field, rating notes, and assignments.

What we are doing

We take a real workflow from our own work, write it as a prompt, run it in Codex 5.5 (Extra High), and rate how it did. The client is OpenAI and the tool is Codex. This is confidential: if asked, we say we provide expertise to improve models for a top AI lab. We never name the client.

Domain is open: coding, sales, lead management, hiring, delivery, any real workflow your role involves.

Two important points from micro1:

- **The prompt only ever runs on your own workspace.** It does not have to be fully self-contained. You can reference your real files, lists, and apps by name and assume the model will access and understand them. You do not need to link or upload them.
- The orientation they want is "do this real thing in my real environment." If you want a reference for the style, the public OSWorld eval set is similar in spirit, though their tasks are easier and more click-focused than ours.

Two stages: first you define and submit a prompt, then after approval there is a separate task where you rate the run in detail. About 1 to 2 hours per full cycle.

The rules that can get you removed, plus the one the reviewer flagged

1. Do not use any AI to write your prompt, context, notes, or ratings. AI is only for running the test. Grammar and spelling check is the only exception, and do not paste its rewrite.
 2. The prompt runs on your real workspace, so reference your real files and apps by name. Just make sure the prompt text and any uploads contain no confidential data, client details, or credentials. If a workflow cannot be described without exposing confidential data, use a scrubbed or fictional stand-in.
 3. Do not name the client.
 4. India is fine to work from.
 5. Do not polish. Write it the way you message a colleague. Casual is preferred. Do not make it formal.
 6. **Write the prompt as instructions to another person. No "I" statements.** Write "Review the records", "Create a folder", "Update the list", not "I review", "I create", "I update". The context and environment fields can still be first person; only the workflow prompt must read as instructions.
-

The bar: what gets approved (and what to actually do)

Each point below is a thing to do, not just a label to remember.

- **Use at least two tools.** Pick a workflow where the agent reads information from one app (a list, sheet, doc, or repo) and then does something in a different app (updates a record, sends a message, creates and links a file). A workflow that lives in one app only will be rejected.
- **End in a real action, not just a document.** Make sure the final step changes something: a record updated, a message sent, a ticket created, a file written back. If the only output is a written summary or report, it scores low. Add a concrete action in a real app.
- **Write it as instructions, not "I" statements.** Phrase every step as a command to the agent: "Review...", "Create...", "Update...". Do not write "I review" or "I create".
- **Name everything the agent needs.** Write the exact list, sheet, folder, repo, and field names so someone with access to your workspace could start without asking you. You do not need to link or upload them. Compare "find the value of my 401k in Fidelity" (clear enough to act on) with "find the value of my stock" (too vague). If the agent is updating an existing system, name the fields being changed. If it is creating something new (a sheet, project, or board), you do not have to design the full structure; just state the minimum information each item must contain.
- **Say what to produce, never the steps to get there.** Tell the agent the goal and the required output, but do not write "first do X, then Y, then Z". Let it pick its own method. If your prompt reads like a step-by-step recipe, delete the steps and keep only the goal and the required result.
- **Make it hard enough that the model cannot do it well.** After you run it in Codex, fill the Feather field "Rate the experience and outcome", where 1 means the model failed or did the task badly and 7 means perfect. We want a low score, around 1 to 3, because that means the prompt was hard enough that the model could not do it well. If Codex scores 5 or higher, it did the task well, so the task is too easy: add a harder decision, an edge case, or another step, then test again. This is a different field from Workflow difficulty, explained just below.
- **Pick something you actually do regularly.** Choose a real, repeating task from your week, not a one-off or an experiment.
- **Keep client data and credentials out of the text.** Reference your real files by name, but never paste real client details, names, or passwords into the prompt or uploads.

Be specific enough to avoid a revision

This is where almost every prompt gets a revision. The complexity is usually fine; the notes are about pinning down specifics. Referencing your workspace by name is fine, but make each of these unambiguous.

- **Bound any "review" or "search" step.** Do not say "review everything available". State the limits: which folder, labels, channels, and accounts, the date range, and which records, clients, or keywords to include.

- **Name the exact account, property, or file when you could have more than one.** Not just "GA4" or "the sheet". Name the specific website, GA4 property, Search Console property, Clarity project, folder, or workspace.
- **Give the model the reference sources for context.** If the output should reflect your values, service area, or selling points, name or link the source (your website, Google Business Profile, a values doc) so it uses the right information.
- **Say what to extract from each input.** If the model must pull specific data first (an invoice date, number, and amount; a case status and owner), name those fields explicitly.
- **Define relative time ranges with exact date logic.** Replace "last 30 days" with logic anchored to the run date, for example "the 30 days ending yesterday" and "the 30 days before that".
- **Say update-in-place or create-new.** State whether to update an existing system or sheet in place, or create a new consolidated one.
- **Specify the output format.** For a created file, give the location, the naming or filename convention, sharing, and any filters. If you define the structure, give the exact columns or headers, date formats, and a sample row. Or leave the structure open to test the model's judgment, stating only the minimum each item must contain (see the balance section below).
- **Define the judgment criteria.** When the model must filter, qualify, or decide, say what "good" or "qualifying" means, for example which leads count and which to exclude.
- **For messages and invites, say draft or live.** State whether a Gmail draft stays unsent, the recipients, and the subject format. For Calendar, state the attendees, target calendar, scheduling rules, and duration, and whether it is a draft or a live invite. Include a link to any file you created.

Balance: be specific, but leave the judgment open

The specificity above has a flip side. Real work handed to a capable junior or intern still leaves room for their judgment. The goal is not to remove all judgment, it is to remove the ambiguity that would make two reviewers score the same run differently. So pin down what affects scoring, and leave open what tests the model's thinking.

Leave open (this is what the eval is testing): how the model organizes the work, the decisions it makes, and the structure it builds on its own. For example: the exact layout or template of a ticket, any extra fields beyond the ones you require, how it groups or orders tasks, its internal reasoning, and how it phrases a summary. Instead of defining every Jira field, require each ticket to contain a few concrete fields (for example the Task ID, the recommended action, and a developer action plan), and let the model add whatever else it judges useful.

Make the minimum concrete. When you leave a structure open, still name the few fields each item must have, then invite the model to add more. A vague minimum like "the task details" or "related details" reads as underspecified to both the validator and reviewers, and they will push you to specify the whole structure. A concrete minimum (a handful of named fields) plus "set up the rest however works best" passes cleanly and still tests the model's judgment on everything beyond the minimum. The goal is a clear minimum, not a full template.

Define explicitly (this is what keeps scoring consistent): anything where different reasonable interpretations would change the output or make the review subjective. For example: the rule for "ready to execute" versus "needs clarification"; research boundaries (which sources, how fresh, whether citations are needed, and when to research at all); the maximum number of items or batch size per run; whether the model may change code or only review it; the completion criteria; and the success or failure conditions.

The test to apply to any detail: would you normally expect a capable junior employee to decide this on their own? If yes, leave it open. If different reasonable choices would materially change the evaluation or the score, define it.

On reviewer feedback: the reviewers' suggestions are suggestions, not hard rules. If adding a specific detail would make the task less like how it would really be assigned, it is fine to leave it open and say why in your notes. A reviewer confirmed this directly: leaving a created structure (like a Jira ticket layout) open is fine when that is how you would authentically assign the work to a colleague, and you can push back on a suggestion if it would compromise the prompt's authenticity.

When the validator and a reviewer disagree: the validator is an LLM and leans toward more specificity, so it may still flag an open structure as a Rule 4 or Rule 9 fail even when you intentionally left it open with a minimum stated. That is a false positive. Trust the reviewer's guidance, keep the structure open, and disregard the flag. Still apply any fixes that affect scoring (decision rules, research scope, batch limits, completion criteria, draft or live).

Applied to the SEO workflow: leave the tracker's ticket structure and extra fields open; define the "ready to execute" versus "needs clarification" rule, the research boundaries, the maximum number of SEO tasks per run, and whether the model may change code or only recommend.

The two 1-to-7 fields in Feather, do not mix them up

Feather has two different 1-to-7 ratings, and at "1" they mean opposite things. This is the easiest thing to get backwards.

- **Workflow difficulty: how hard the workflow is to do.** The field asks "where 1 is extremely easy and 7 is extremely difficult". Just report yours honestly, based on the effort and thought the task takes you. There is no target here; a real, thought-heavy workflow tends to be on the higher side, but you simply state the truth.
- **Rate the experience and outcome: how well Codex did.** The field asks for a score "from 1 (horrible / completely unsuccessful) to 7 (perfect execution and experience)". Here we want a **low** score, around 1 to 3, because a low score means the model could not do the task well, which is exactly what a good eval prompt should cause. If Codex scores 5 to 7, it did the task well, so the prompt is too easy and needs more difficulty.

In short: rate the workflow's own difficulty **honestly**, and aim for the Codex outcome score to come out **low**. Two different fields, and a "1" means opposite things in each: an easy workflow versus a model that completely failed.

Why prompts get rejected, and how we avoid each

1. **Written in the first person ("I do this").** Fix: rephrase every step as a command. This is the most common note so far.
 2. **Only makes a document.** Fix: add a real action in a second app, such as updating a record or sending a message.
 3. **Too easy.** Fix: keep the judgment and the messy edge cases in, and confirm the Codex outcome score comes out low (around 1 to 3 on "Rate the experience and outcome", meaning the model could not do it well).
 4. **Too vague to act on.** Fix: name the exact files, lists, and fields. For a newly created system, name the minimum information each item must contain rather than the full structure.
 5. **Confidential data in the text or uploads.** Fix: reference by name only; never paste client data or credentials.
 6. **Same as a teammate, or your own task with just different inputs.** Fix: choose a meaningfully different task.
 7. **One app only.** Fix: one app to read from, a different app to act in.
 8. **Reads as AI-written.** Fix: write it yourself, casually.
-

STEP 1: List your tasks (keep it simple)

Do not try to write a workflow yet. Each person lists what they actually do, grouped by how often. Plain words are fine.

Fill this in:

- **Every day:** things you do almost every working day
- **A few times a week:**
- **Weekly:** once-a-week tasks
- **Monthly or a few times a month:**
- **When it comes up:** as-needed tasks

For each item, add two quick notes:

- which apps or tabs you open for it
- what you create, update, or send at the end

The good candidates touch two or more apps and end in you changing or sending something. Once everyone has their list, bring the answers together and we turn each person's strongest task into a personalized prompt, checking that no two people get the same idea. This is the first practice round; once the team is comfortable, we move to a faster way of finding ideas.

STEP 2: Turn one task into a prompt, written as instructions

This is a real prompt one of us submitted that the reviewer praised, shown before and after the one fix he asked for. Use it as the model.

How it came out in the first person (what got flagged):

I daily review all records in the Microsoft list "Lead Tracker" where the status is "Meeting Scheduled". For every record I use the record info and the URLs to prepare for the client meeting, then I create a folder in Google Drive, I create a Google Doc per project, I check when information is insufficient and I send a follow-up, then I update the list.

The fix: rewrite it as instructions to someone else. Drop every "I".

The approved version (instructions, specific, open-ended on how):

Use the records in the Microsoft list "Lead Tracker" where the status field is "Meeting Scheduled". For every record, use the information in the record and all URLs stored in the project-related URL fields (Project URL, Conversation URL, Shared Documents URL, and any other URLs attached to the record) to prepare the project for an upcoming client meeting. Create a folder in Google Drive under "Meeting Preparation" using today's date. In that folder, create one Google Doc per project using the project name as the title. Each doc must contain: Client Background, Project Requirements, Current Project Status, Open Questions, Risks or Concerns, Recommended Discussion Topics, and a Meeting Readiness Assessment classified as Ready for Meeting, Needs Clarification, or High Risk. Where the available information is insufficient for an effective meeting, send a follow-up message through the communication platform in the record (Lead source column: Upwork or Freelancer). The message must clearly identify the missing information, explain why it is needed for meeting preparation, and request the required clarification in a professional manner. After processing each record, update the Microsoft list "Lead Tracker" by adding the Google Doc URL to the "Meeting Doc" column.

Why it passes: it reads from a Microsoft List and linked URLs, acts in Google Drive and on Upwork or Freelancer and writes back to the List, names every field, leaves the method open, and the synthesis plus readiness judgment plus the conditional follow-up make it genuinely hard. It references the real workspace by name without uploading anything, and the text holds no client data. The only change from the first draft was removing the "I" statements.

Every other Feather field comes from your Step 1 notes and your setup. There are more fields than people expect, including one for the environment, so use the full list in the reference section.

STEP 3: Validate before submitting

Run this in plain ChatGPT or Claude on your **workflow description / prompt**. It checks your prompt against the nine rules, writes each fix out with brackets for the values you fill in, converts

any relative dates to exact calendar dates, flags anything that reads AI-written and rewrites it to sound human, and returns a full fixed version of your prompt with clear markers. Keep the workflow and the voice yours; the AI-written check is there to keep the final sounding like you wrote it. Fix what it flags, then submit.

You are reviewing a workflow prompt I wrote for an AI workflow-evaluation project. The prompt only ever runs on my own workspace, so it may reference my real files and apps by name. Check it against the 9 rules below and return the result in the exact format shown. Keep everything short, plain, and easy to read.

Most important for this review: if my prompt creates a new system (a sheet, ticket, board, or project) and says what each item must AT LEAST contain, that is a stated minimum and rules 4 and 9 PASS. Do not fail those rules for an open structure, do not ask for a fuller field list, and do not prescribe one in the fix. Leaving structure open to test the model's judgment is a valid, authentic choice. The only structure details you may still require are ones that change scoring (decision rules, research scope, batch limits, completion criteria, draft or live).

While reviewing:

- Whenever my prompt uses a relative time range like "last 30 days" or "this quarter", convert it to exact calendar dates based on today's date, and use those dates in the fix.
- For each broken rule, write the fix line out in full, using square brackets [like this] only for the specific values I must fill in (property names, folders, people).
- Flag any sentence that reads as AI-written, overly polished, or too formal. Rewrite each of those to sound like a person wrote it casually, and apply that rewrite in the final prompt.
- Balance test, apply this before failing rule 4 or rule 9 for any missing detail. Ask whether different reasonable interpretations of the detail would materially change the output or make scoring subjective. If yes, it should be defined (for example: the rule separating two statuses, research scope, batch limits, completion criteria, whether code may be changed). If a capable junior employee would normally decide it on their own (for example: the structure or extra fields of a sheet, ticket, or board the model creates, or how it organizes the work), it is fine to leave it open. Do NOT fail a prompt for an intentionally open structure when the minimum required information is stated.

The 9 rules:

1. Written as instructions to another person, no first-person "I" statements.
2. Uses at least two apps: reads from one app and takes a real action in a different app. A document or summary alone does not count.
3. Ends in a real change (an updated record, a sent message), not just a file.
4. Names the exact resources: the specific list, sheet, repo, doc, folder, channel, account, or property when more than one could exist. For an existing system, names the fields changed. For a new system, states the minimum information each item must contain; leaving the rest of the structure open to test the model's judgment is fine and PASSES. For example, "configure the Jira project, each ticket must at least contain the Task ID

and related task details" passes, even without a full field list.

5. Says what to produce, not the step-by-step method.
6. Hard enough that a top model would not do it perfectly.
7. A real, recurring task, not experimental.
8. No confidential client data or credentials in the text.
9. Specific enough to avoid a revision, applying the balance test above. Require a detail only where a different reasonable interpretation would change the output or scoring: bounds any "review/search" step (folder, labels, channels, date range, records, keywords); names the exact account or property; gives reference sources; says what to extract from inputs; defines relative dates with exact dates; says update-in-place or create-new; gives the output's destination (location, naming, sharing, filters); defines the judgment criteria when the model must filter or rank, the decision rules, the completion criteria, and any batch limits; and for messages or invites says draft or live, recipients or attendees, subject, and a link to any created file. The structure and extra fields of something the model creates may be left open if the minimum information is stated.

Return exactly this, in plain language:

VERDICT: READY TO SUBMIT or NOT READY. If not ready, the single biggest reason in one line.

CRITERIA CHECK: List each rule 1 to 9 with PASS or FAIL and a few words why.

FIXES: For every FAIL, write two lines:

- Wrong: quote the exact line from my prompt, or "missing" if nothing is there.
- Fix: the line written out, with [brackets] for the values I fill in. Convert any relative dates to exact dates based on today.

AI-WRITTEN CHECK: Quote any sentences that read as AI-written, overly polished, or too formal. If there are none, write "reads human". Whatever you quote here, rewrite to sound casual and human in the final prompt.

WORKFLOW DIFFICULTY: A number 1 to 7 for how hard the workflow is for a person (1 extremely easy, 7 extremely difficult), with one line of why. This is the Feather "Workflow difficulty" field. Tell me to set my own honest figure.

EXPECTED OUTCOME SCORE: A number 1 to 7 for how well a top model would do (1 it fails, 7 perfect). This is the Feather "Rate the experience and outcome" field. We want a low score, 1 to 3. If you expect 5 to 7, say it is too easy and name one thing to make it harder.

FULL FIXED PROMPT: My original prompt with every fix applied. Mark removed text with ~~double tildes~~ and added or rewritten text with {{ double curly braces }}. Do not use bold. Keep my original wording wherever it already sounds human. Use [brackets] for the values I fill in. Rewrite any AI-sounding sentence to sound casual and human, shown as ~~old text~~ {{ new text }}.

Here is my workflow prompt:
[paste here]

In the full fixed prompt, removed text is struck through and additions sit inside `{{ }}`, so you can see every change in any plain text editor. Anything in `[ ]` is a value you replace with your own.

Short example of what it returns (for a weak draft like "I review the bugs in the sheet and write a summary report"):

VERDICT: NOT READY. It only creates a document and never acts in a second app.
CRITERIA CHECK: Rule 1 FAIL (uses "I"); Rule 2 FAIL (one app only); Rule 3 FAIL (ends in a summary); Rules 4, 5, 7, 8 PASS; Rule 6 FAIL (too easy); Rule 9 FAIL (no specifics).
FIXES: Wrong: "I review the bugs in the sheet and write a summary report." Fix: "Review the bugs in the [bug sheet name], then create an issue in [GitHub repo] for each real bug with [the fields each issue needs], and update the Status column in the sheet."
AI-WRITTEN CHECK: reads human.
WORKFLOW DIFFICULTY: 2 of 7. Reading a sheet and writing a summary takes little effort. Set your own honest figure.
EXPECTED OUTCOME SCORE: 6 of 7, too easy. Add a real decision, like deduplicating bugs and assigning a priority and owner to each.
FULL FIXED PROMPT: ~~I review the bugs in the sheet and write a summary report.~~
`{{ Review the bugs in the [bug sheet name], deduplicate them, then create an issue in [GitHub repo] for each real bug with [the fields each issue needs], set a priority of [your levels], assign an owner from [your team list], and update the Status column in the sheet. }}`

When your verdict reads READY TO SUBMIT and every criterion is PASS, you are good to move on.

STEP 4: Run it

- Use Codex 5.5 on **Extra High**. Token cost is billed to micro1, so do not worry about usage.
- Connect the apps you need. If an app has no connector, use **browser control**: you log in yourself, then let the agent drive. Never share passwords or codes.
- Codex can also work in your **local setup**: open your project, change code, run tests, and commit and push with git, then update a tracker or sheet. That combination is a strong workflow.
- Paste the prompt as-is. Do not add framing like "please test this."
- **Actually run it and confirm it is hard enough.** The reviewer asked us to test in Codex and make sure it is sufficiently challenging. On the "Rate the experience and outcome" field (1 = horrible, 7 = perfect), we want a low score, around 1 to 3, meaning the model could not do it well. If Codex does it well or perfectly (5 to 7), add difficulty before submitting.
- **Run the same prompt in Claude too** and note where Codex falls short. The goal is getting Codex level with Claude.

- Copy the **session ID** from Codex into Feather so the client can inspect the run.
-

STEP 5: Fill every field, mark Completed, and ping

- Fill in **every Feather field**, not just the prompt. The reviewer asked us to complete the whole task.
 - Tick the confidentiality box only after confirming no real client data or credentials are in the text.
 - Mark the task **Completed** in the top right when done.
 - Drop the **Feather link in the micro1 companies Slack workspace and tag all three** leads (Solomon, Christian, Camilo). Review target is within 24 hours, faster if you tag.
 - Use only the **micro1 companies** Slack workspace. Ignore the experts workspace and do not use the OpenAI workspace directly.
 - Set your Slack **display name to your real first name**.
-

Reference: the tools you can use

Do not limit yourself. Connector if it exists, browser control if not, local if it lives on your laptop.

- **Local development:** Codex can open your project, edit code, run tests, and run git commit and push. Pair with a cloud action (update Jira, a Google Sheet, a board, or Slack) for a strong workflow.
- **GitHub:** issues, PRs, labels, comments, boards
- **Google Sheets, Docs, Drive, Calendar, Gmail:** trackers, specs, files, scheduling, messages
- **Microsoft Lists, Jira, Notion, Slack:** records, tickets, docs, team messages
- **Browser control:** for anything without a connector, including LinkedIn, Upwork, Freelancer, Microsoft Lists, a careers-page admin panel, or any web tool. You log in, the agent acts.

The only constraints: keep confidential data and credentials out of the prompt text, and never hand over passwords or login codes.

Reference: real SME examples by role

Grounded in how our agency runs. Write the actual prompt as instructions (no "I"). One line each: the real task, then read from to act in, with the judgment that makes it hard.

Team Lead

- Performance and hours review: read the team's timesheet and their closed tickets, summarize each person's output against hours, flag anyone overloaded or with low

throughput, draft individual notes in a Doc, set 1:1 reminders. Hard: fair read of output versus hours.

- New requirement to timeline: read the requirement the PM shared, break it into tasks, build a timeline with owners and dates, create tickets, post a summary to the team. Hard: estimating and sequencing, ambiguous requirements.
- Monthly client sync prep: read the project tracker and recent activity, draft a client update and agenda with open questions and risks, update the tracker, schedule the meeting. Hard: deciding what to surface.

PM or department head

- Meeting preparation: the worked example in Step 2.
- Job-post vetting: read the BDE team's shared job posts, judge each against our tech stack, mark go, no-go, or explore with a reason, and message BDE on which to pursue. Hard: borderline stacks.
- New lead intake: read incoming leads, qualify by budget and fit, update status and follow-up fields, draft a first response, route to the right department. Hard: qualification calls.

BDE team

- Bid tracker update: check recent discussions on Upwork or Freelancer (browser control), update each bid's status in the bids sheet, flag the ones that went quiet, draft follow-up messages. Hard: telling real interest from a polite no.
- LinkedIn outreach: find relevant people on LinkedIn (browser control), pick those likely to need custom development or a dedicated developer, draft a personalized connection note for each, log them in an outreach sheet. Hard: relevance and non-generic notes. Draft, do not auto-send.

HR and recruiter

- Job posting pipeline: from the PM's required-talent list, draft a job description per role, brief the designer for the social image, post to the job portals and update the careers page (browser control), log each opening's status. Hard: tailoring each JD and choosing channels.
- Candidate triage: read applications, screen against requirements, update status and next-step fields, draft interview-invite or rejection emails, schedule screens. Hard: screening judgment.

Engineering

- Local change to PR: in the local repo, implement the change described in a GitHub issue, run the tests, commit and push to a branch, open a PR, move the issue and board card to "In Review". Hard: getting the change correct and tests passing.
- Bug triage: from a bug sheet with duplicates and vague entries, dedupe, create issues with repro and priority, update the sheet, draft a Slack top-5. Hard: dedup and severity.
- Incident to actions: from an incident write-up, create remediation issues with owners, log them in a tracker, schedule a review. Hard: turning narrative into owned actions.

Reference: if you cannot point at a real workspace

Most of us can reference our real setup, which is preferred. Use this only if your workflow would expose confidential data or you do not have the real data on hand. Build a realistic fake scenario, then run the agent on it. The workflow must still be one you genuinely do.

- 10 to 20 rows, or 5 to 8 items
 - Plant the messy edge cases a real version has: a cold lead, a borderline job post, a duplicate bug, an over-capacity person, an ambiguous requirement. Without these the model scores high and the task is rejected as too easy
 - Invented names, demo accounts, a personal Google account
-

Reference: the worked example, every Feather field

The real meeting-preparation workflow from Step 2, with every field filled. Use your own real numbers for the metadata.

Workflow description / prompt (instructions, no "I")

Use the records in the Microsoft list "Lead Tracker" where the status field is "Meeting Scheduled". For every record, use the information in the record and all URLs stored in the project-related URL fields (Project URL, Conversation URL, Shared Documents URL, and any other URLs attached to the record) to prepare the project for an upcoming client meeting. Create a folder in Google Drive under "Meeting Preparation" using today's date. In that folder, create one Google Doc per project using the project name as the title. Each doc must contain: Client Background, Project Requirements, Current Project Status, Open Questions, Risks or Concerns, Recommended Discussion Topics, and a Meeting Readiness Assessment classified as Ready for Meeting, Needs Clarification, or High Risk. Where the available information is insufficient for an effective meeting, send a follow-up message through the communication platform in the record (Lead source column: Upwork or Freelancer). The message must clearly identify the missing information, explain why it is needed for meeting preparation, and request the required clarification in a professional manner. After processing each record, update the Microsoft list "Lead Tracker" by adding the Google Doc URL to the "Meeting Doc" column.

Local professional environment and resources the agent needs (the field people miss)

This runs on my work setup. Lead records live in a Microsoft List called "Lead Tracker" with status, lead source, and several project URL fields. The project context sits behind those URLs, in Upwork or Freelancer conversation threads and shared documents. Output goes to a Google Drive folder, and follow-up messages go through Upwork or Freelancer depending on the record's lead source.

Applications required: Microsoft Lists, Google Drive and Google Docs, Upwork, Freelancer (browser control for the platforms and the List if no connector)

Additional context (why, when, larger workflow)

I do this daily before client meetings, as part of our sales-to-delivery handoff. When a lead reaches "Meeting Scheduled", the project needs a prepared brief so the meeting is productive. It feeds a larger pipeline from lead intake through delivery.

Interim checkpoints / required outputs (the distinct stages, for partial credit)

- Selects only records with status "Meeting Scheduled"
- Reads and uses the record fields and the linked URLs
- Creates the dated folder in Google Drive under "Meeting Preparation"
- Produces one Google Doc per project with all required sections and a readiness classification
- Sends a clear follow-up on the correct platform only where information is insufficient
- Writes the Google Doc URL back into the "Meeting Doc" column

Occupation dropdown: Project Manager / Operations **Occupation and workplace:** Project manager at a software development agency, handling client leads and pre-meeting preparation. (Keep this short and specific: your role, your workplace, and your focus. For example, "Intern at an HVAC company, focused on AI integration across the business.") **Time to complete manually:** 60 minutes (use your real figure) **Times per month:** 20 (use your real figure) **Workflow difficulty (how hard the workflow is to do: 1 extremely easy to 7 extremely difficult):** 5 (use your real figure) **Rate the experience and outcome (how well Codex did: 1 horrible to 7 perfect):** fill after running, for example 3 (a low score is good here, it means the prompt was hard enough that the model could not do it well) **Codex notes:** evidence-based, for example "Created the folder and docs but missed two readiness classifications, and sent a follow-up on a record that had enough information. In Claude the follow-up logic was better but the docs were thinner. [Paste exact error text if anything failed.]" **Confidentiality checkbox:** tick after confirming the prompt text holds no client data.

Reference: rating notes with evidence

For each app state: did the connector exist, did it connect, did the model complete the action, and quote any exact error text. Never write only "worked well." If two models tie, write the ranking as a tie (for example $A > C = B$, meaning A is best and C and B are equal). Add the side-by-side Claude observation where useful.

Reference: volume, diversity, and assignments

About 3 solid prompts per person, up to 5, all meaningfully different. Same workflow with different inputs does not count as new.

Starting split so no two overlap (adjust to who does what):

- Person A: meeting preparation (worked example)

- Person B: new requirement to timeline
 - Person C: bug triage or local change to PR
 - Person D: bid tracker update
 - Person E: performance and hours review
-

Pre-submit checklist

- ☐ Picked a real, repetitive task from your Step 1 list
- ☐ Prompt is written as instructions, with no "I" statements
- ☐ Reads from one app and acts in a different app, with a real action
- ☐ Named every list, file, and field, referencing your real workspace
- ☐ Says what to produce, not the step-by-step method
- ☐ No client data or credentials in the text
- ☐ Pinned down the specifics reviewers ask for: bounded any "review/search" step, named the exact account or property, gave reference sources, said what to extract, exact date logic, update-in-place or create-new, output format (columns, date formats, sample row, filename), judgment criteria, and draft-or-live for messages and invites
- ☐ Ran the validation prompt and fixed every flag
- ☐ Ran it in Codex on Extra High and the outcome score came out low (around 1 to 3 on "Rate the experience and outcome", meaning the model could not do it well)
- ☐ Ran it in Claude for comparison
- ☐ Copied the session ID into Feather
- ☐ Filled in every Feather field, including the environment field
- ☐ Wrote rating notes with evidence
- ☐ Marked the task Completed and posted the Feather link in the micro1 companies Slack, tagging all three leads